

Programação

Dispositivos Móveis

aula 02

API Level

- Cada versão do Android possui uma API Level.
- É importante para referenciar as versões do Android.
- A API Level é um valor inteiro que identifica uma versão do Android.
- A plataforma Android fornece uma estrutura de API que os aplicativos podem usar para interagir com o Sistema Android.

A estrutura API consiste em:

- Conjunto de pacotes e classes;
- Conjunto de elementos e atributos XML para declarar um manifest file (arquivo de configuração da aplicação no Android);
- Conjunto de elementos e atributos XML para declarar e acessar recursos;
- Conjunto de intenções ("intents");
- Conjunto de permissões que os aplicativos podem solicitar, bem como a autorização incluída no sistema;
- Cada versão sucessiva da plataforma Android pode incluir atualizações e com a API Level é possível identificar sua utilização no mesmo.

Máquina Virtual Dalvik

- Aplicações escritas em Java são compiladas em bytecodes Dalvik e executadas usando a Máquina Virtual Dalvik.
- Máquina virtual especializada desenvolvida para uso em dispositivos móveis
- Baseada em registradores
- Otimizada para utilizar pouca memória
- Permite que múltiplas instâncias da MV rodem ao mesmo tempo, deixando para o SO o isolamento de processos, o gerenciamento de memória e o suporte a threading.

Principais Classes

- Activity
- View
- Service
- Content Provider
- BroadcastReceiver
- Intent

Outras Classes/Funcionalidades

- Notification
- AlarmManager
- Handler, Threads e AsyncTask
- Banco de dados (SQLite)
- Mapas e GPS
- Http, sockets e Web Services
- SMS
- Google Cloud Messaging
- Reconhecimento de Gestos
- Sensores
- Bluetooth, OpenGL, NDK ...

- Aplicativos para Android são criados como uma combinação de componentes distintos que podem ser invocados individualmente.
- Por exemplo: uma *atividade* individual fornece uma única tela para uma interface do usuário e um *serviço* realiza o trabalho no segundo plano de forma independente.

- Em um componente, é possível iniciar outro componente usando uma *intent*.
- É possível iniciar um componente em um aplicativo diferente, como uma atividade em um aplicativo de mapas para exibir um endereço.
- Esse modelo fornece vários pontos de entrada para um único aplicativo e permite que qualquer aplicativo se comporte como o “padrão” de um usuário para uma ação que outros aplicativos podem invocar.

- O Android fornece uma estrutura de aplicativo adaptativa que permite fornecer recursos exclusivos para diferentes configurações de dispositivos.
- Por exemplo: é possível criar diferentes arquivos XML de layout para diversos tamanhos de tela e o sistema determina qual layout deverá aplicar com base no tamanho da tela do dispositivo atual.

- Você pode consultar a disponibilidade dos recursos do dispositivo em tempo de execução se qualquer recurso do aplicativo exigir hardware específico, como uma câmera.
- Se necessário, também é possível declarar recursos que o aplicativo exige para que mercados como a Google Play Store não permitam a instalação em dispositivos que não sejam compatíveis com aquele recurso.

- Os aplicativos do Android são programados em linguagem de programação Java.
- As ferramentas do Android SDK compilam o código — em conjunto com todos os arquivos de dados e recursos — em um APK — um *pacote Android*, que é um arquivo de sufixo “.apk”.
- Os arquivos de APK contêm todo o conteúdo de um aplicativo do Android e são os arquivos que os dispositivos desenvolvidos para Android usam para instalar o aplicativo.

- O sistema operacional Android é um sistema Linux multiusuário em que cada aplicativo é um usuário diferente.
- O sistema atribui a cada aplicativo um ID de usuário do Linux exclusivo.
- Este ID é usado somente pelo sistema e é desconhecido para o aplicativo.
- O sistema define permissões para todos os arquivos em um aplicativo, de modo que somente o ID de usuário atribuído àquele aplicativo pode acessá-los.

- Cada processo tem sua própria máquina virtual (VM), portanto, o código de um aplicativo é executado isoladamente de outros aplicativos.
- Cada aplicativo é executado no próprio processo do Linux.
- O Android inicia o processo quando é preciso executar algum componente do aplicativo; em seguida, encerra-o quando não mais é necessário ou quando o sistema precisa recuperar memória para outros aplicativos.

- O sistema Android implementa o *princípio do privilégio mínimo*.
- Por padrão, cada aplicativo tem acesso somente aos componentes necessários para a execução do seu trabalho e nada mais.
- Isso cria um ambiente muito seguro em que o aplicativo não pode acessar partes do sistema para o qual não tem permissão.

- Existe uma maneira de um aplicativo compartilhar dados com outros aplicativos e acessar serviços do sistema:
- É possível fazer com que dois aplicativos compartilhem o mesmo ID de usuário do Linux, caso em que eles são capazes de acessar os arquivos um do outro.
- Para preservar os recursos do sistema, os aplicativos com o mesmo ID de usuário também podem ser combinados para serem executados no mesmo processo Linux e compartilharem a mesma VM (também é preciso atribuir o mesmo certificado aos aplicativos).

- Um aplicativo pode solicitar permissão para acessar dados de dispositivo como contatos do usuário, mensagens SMS, o sistema montável (cartão SD), câmera, Bluetooth etc.
- O usuário precisa conceder essas permissões de forma explícita.

- Componentes de aplicativo são os blocos de construção de um aplicativo Android.
- Cada componente é um ponto diferente pelo qual o sistema pode entrar em seu aplicativo.
- Nem todos os componentes são pontos de entrada reais para o usuário e alguns dependem uns dos outros.
- Cada componente existe como uma entidade independente e desempenha uma função específica.
- Cada componente é um bloco de construção exclusivo que ajuda a definir o comportamento geral do aplicativo.

- Há quatro tipos de componentes de aplicativo. Cada tipo tem uma finalidade distinta e tem um ciclo de vida específico que define a forma pela qual o componente é criado e destruído:
 - Atividades
 - Serviços
 - Provedores de Conteúdo
 - Receptores de transmissão

- Atividades representam uma tela única com uma interface do usuário.
- Por ex: um aplicativo de e-mail pode ter uma atividade que mostra uma lista de novos e-mails, outra atividade que compõe um e-mail e outra ainda que lê e-mails.
- Embora essas atividades funcionem juntas para formar uma experiência de usuário coesa no aplicativo de e-mail, elas são independentes entre si.
- Portanto, um aplicativo diferente pode iniciar qualquer uma dessas atividades (se o aplicativo de e-mails permitir).

- Por exemplo, um aplicativo de câmera pode iniciar a atividade no aplicativo de e-mail que compõe o novo e-mail para que o usuário compartilhe uma foto.
- Uma atividade é implementada como uma subclasse de "Activity"

- Serviços são componentes executados em segundo plano para realizar operações de execução longa ou para realizar trabalho para processos remotos.
- Eles não apresentam uma interface do usuário.
- Por exemplo, um serviço pode tocar música em segundo plano enquanto o usuário está em um aplicativo diferente ou buscar dados na rede sem bloquear a interação do usuário com uma atividade.
- Outro componente, como uma atividade, pode iniciar o serviço e deixá-lo executar ou vincular-se a ele para interagir.
- Um serviço é implementado como uma subclasse "Service"

- Provedores de conteúdo gerenciam um conjunto compartilhado de dados do aplicativo.
- É possível armazenar os dados no sistema de arquivos, em um banco de dados SQLite ou em qualquer local de armazenamento persistente que o aplicativo possa acessar.
- Por meio do provedor de conteúdo, outros aplicativos podem consultar ou até modificar os dados (se o provedor de conteúdo permitir).
 - Por exemplo, o sistema Android oferece um provedor de conteúdo que gerencia os dados de contato do usuário.
 - Qualquer aplicativo com as permissões adequadas pode consultar parte do provedor de conteúdo para ler e gravar informações sobre uma pessoa específica.

- Os provedores de conteúdo são úteis para ler e gravar dados privados no aplicativo e não compartilhados.
- Por exemplo, o exemplo de aplicativo NotePad usa um provedor de conteúdo para salvar notas.
- Um provedor de conteúdo é implementado como uma subclasse de “Content Provider” e precisa implementar um conjunto padrão de APIs que permitem a outros aplicativos realizar transações.

- Receptores de transmissão são componentes que respondem a anúncios de transmissão por todo o sistema.
- Muitas transmissões se originam do sistema — por exemplo, uma transmissão que anuncia que uma tela foi desligada, a bateria está baixa ou uma tela foi capturada.
- Os aplicativos também podem iniciar transmissões — por exemplo, para comunicar a outros dispositivos que alguns dados foram baixados no dispositivo e estão disponíveis para uso.

- Embora os receptores de transmissão não exibam nenhuma interface do usuário, eles podem criar uma notificação na barra de status para alertar ao usuário quando ocorre uma transmissão.
- Mais comumente, no entanto, um receptor de transmissão é somente um "portal" para outros componentes e realiza uma quantidade mínima de trabalho.
- Por exemplo, ele pode iniciar um serviço para executar um trabalho baseado no evento.
- Os receptores de transmissão são implementados como subclasses de "Broadcast Receiver" e cada transmissão é entregue como um objeto "Intent".

- Um aspecto exclusivo do projeto do sistema Android é que qualquer aplicativo pode iniciar um componente de outro aplicativo.
- Por exemplo, se você quiser que o usuário capture uma foto com a câmera do dispositivo, provavelmente haverá outro aplicativo que faz isso e seu aplicativo poderá usá-lo, ou seja, não será necessário desenvolver uma atividade para capturar uma foto.
- Não é necessário incorporar nem mesmo vinculá-lo do aplicativo da câmera ao código.
- Em vez disso, basta iniciar a atividade no aplicativo da câmera que captura uma foto.

- Quando concluída, a foto até retorna ao aplicativo em questão para ser usada. Para o usuário, parece que a câmera é realmente parte do aplicativo.
- Quando o sistema inicia um componente, ele inicia o processo daquele aplicativo (se ele já não estiver em execução) e instancia as classes necessárias para o componente.
- Por exemplo: se o aplicativo1 iniciar a atividade no aplicativo2 da câmera que captura uma foto, aquele aplicativo1 é executado no processo que pertence ao aplicativo2 da câmera e não no processo do aplicativo1.

- Portanto, ao contrário dos aplicativos na maioria dos outros sistemas, os aplicativos do Android não têm nenhum ponto de entrada único (não há a função `main()`, por exemplo).
- O sistema executa cada aplicativo em um processo separado com permissões de arquivo que restringem o acesso a outros aplicativos.
- Portanto, para ativar um componente em outro aplicativo, é preciso enviar uma mensagem ao sistema que especifique o "*intent*" de iniciar um componente específico.
- Em seguida, o sistema ativa o componente.

- Os componentes — atividades, serviços e receptores de transmissão — são ativados por uma mensagem assíncrona chamada “intent”.
- Os intents vinculam componentes individuais entre si em tempo de execução (como mensageiros que solicitam uma ação de outros componentes), seja o componente pertencente ao aplicativo ou não.
- O “intent” é criado com um objeto “Intent”, que define uma mensagem para ativar um componente específico ou um tipo específico de componente.

- Para atividades e serviços, os intents definem a ação a executar (por exemplo, "exibir" ou "enviar" algo) e podem especificar a localização dos recursos usados na ação (URI).
- Por exemplo, um intent pode transmitir uma solicitação de uma atividade para exibir uma imagem ou abrir uma página da Web.
- Em alguns casos, é preciso iniciar uma atividade para receber um resultado; nesse caso, a atividade também retorna o resultado em um Intent
 - Por exemplo, é possível emitir um intent para que o usuário selecione um contato pessoal e retorne-o a você — o intent de retorno contém uma URI que aponta para o contato selecionado).

- Para receptores de transmissão, o intent simplesmente define o anúncio que está sendo transmitido
 - Por exemplo, uma transmissão para indicar que a bateria do dispositivo está acabando contém uma string de ação conhecida que indica "nível baixo da bateria").

- O outro tipo de componente, o provedor de conteúdo, é ativado por uma solicitação de um "ContentResolver" e não por "intents".
- O "ContentResolver" (resolvedor de conteúdo) trata todas as transações diretamente com o provedor de conteúdo.
- Um componente para executar transações com o provedor de conteúdo precisa chamar os métodos no objeto "ContentResolver".
- Objetivo: Deixar por segurança uma camada de abstração entre o provedor de conteúdo e o componente que solicita informações.

- Há dois métodos para ativar cada tipo de componente:
- É possível iniciar uma atividade (ou dar-lhe algo novo para fazer) passando uma Intent a `startActivity()` ou `startActivityForResult()` (para que, quando desejado, a atividade retorne um resultado).
- É possível iniciar um dispositivo (ou dar novas instruções a um serviço em andamento) passando uma Intent a `startService()`. Também é possível vincular ao serviço passando uma Intent a `bindService()`.

- Pode-se iniciar uma transmissão passando uma Intent para métodos como `sendBroadcast()`, `sendOrderedBroadcast()` ou `sendStickyBroadcast()`.
- É possível iniciar uma consulta a um provedor de conteúdo chamando `query()` em um `ContentResolver`.